



Prog II:

**Ponteiros Inteligentes
e Estrutura de dados
avancçadas**

Prof^a. Tainá Isabela



Ponteiros inteligentes (smart pointers)

Smart pointers em C++ são classes que encapsulam ponteiros brutos, gerenciando automaticamente a alocação e liberação de memória. Eles ajudam a evitar problemas comuns como vazamento de memória, ponteiros pendentes e uso de memória já libertada, tornando o código mais seguro e robusto.

Ponteiros inteligentes (smart pointers)

Tipos principais de smart pointers:

- **std::unique_ptr**: Possui exclusivamente o objeto apontado. Não pode ser copiado, apenas movido.
- **std::shared_ptr**: Permite múltiplos ponteiros compartilharem a posse do mesmo recurso, usando contagem de referências.
- **std::weak_ptr**: Observa um objeto gerido por `shared_ptr`, mas não participa da contagem de referências, evitando ciclos.

Exemplos:

`unique_ptr`:

```
#include <iostream>
```

```
#include <memory>
```

```
int main() {
```

```
    std::unique_ptr<int> ptr = std::make_unique<int>(42);
```

```
    std::cout << *ptr << std::endl; // Saída: 42
```

```
} // ptr é destruído automaticamente aqui e a memória é libertada
```

Exemplos:

shared_ptr:

```
#include <iostream>
#include <memory>
```

```
int main() {
    std::shared_ptr<int> ptr1 = std::make_shared<int>(10);
    {
        std::shared_ptr<int> ptr2 = ptr1;
        std::cout << ptr1.use_count() << std::endl; // Saída: 2
    } // ptr2 sai de escopo, contagem volta para 1
    std::cout << ptr1.use_count() << std::endl; // Saída: 1
} // ptr1 destruído, memória libertada
```

Exemplos:

weak_ptr:

```
#include <iostream>
```

```
#include <memory>
```

```
int main() {
```

```
    std::shared_ptr<int> sp = std::make_shared<int>(20);
```

```
    std::weak_ptr<int> wp = sp; // wp observa sp, mas não impede a  
    liberação
```

```
    if (auto spt = wp.lock()) { // lock retorna shared_ptr se o objeto ainda  
    existir
```

```
        std::cout << *spt << std::endl; // Saída: 20
```

```
    }
```

```
}
```

Vantagens:

- **Gestão automática de memória:** O objeto é destruído automaticamente quando não há mais ponteiros inteligentes apontando para ele.
- **Segurança contra vazamentos:** Reduz drasticamente o risco de esquecer de libertar memória.
- **Sintaxe semelhante a ponteiros tradicionais:** Operadores * e -> são sobrecarregados, facilitando o uso.

Cuidados com Ponteiros:

- Ponteiros não inicializados podem causar erros graves.
- Liberte sempre a memória alocada dinamicamente com `delete` ou `delete[]`.
- Evite aceder memória já libertada (ponteiros soltos).
- Prefira ponteiros inteligentes (`std::unique_ptr`, `std::shared_ptr`) no C++ moderno para maior segurança.

Resumo – Ponteiros:

- Ponteiros são variáveis que armazenam endereços de memória.
- Permitem manipulação eficiente de arrays, funções e alocação dinâmica.
- Exigem atenção para evitar vazamentos de memória e acessos inválidos.
- São fundamentais para programação eficiente e flexível em C++.

Estruturas de Dados Avançadas em C++: Listas, Filas e Pilhas

As estruturas de dados avançadas são essenciais para a construção de programas eficientes e organizados em C++. Entre as mais utilizadas estão as listas, filas e pilhas, cada uma com características e aplicações específicas.

Listas

- Definição: Estruturas que armazenam elementos de forma sequencial, permitindo inserções e remoções em posições arbitrárias.
- Tipos principais:
 - **Listas simplesmente encadeadas:** cada elemento aponta para o próximo.
 - **Listas duplamente encadeadas:** cada elemento aponta para o anterior e o próximo.
 - **Listas circulares:** o último elemento aponta para o primeiro.
- Vantagens:
 - Flexibilidade para manipular dados dinâmicos.
 - Inserção e remoção eficientes em qualquer posição (especialmente em listas encadeadas).
- Exemplo em C++: A STL oferece a classe **`std::list`**, que implementa uma lista duplamente encadeada.

Listas

Para usar listas, inclua o cabeçalho:

```
#include <list>  
#include <iostream>  
using namespace std;
```

Podemos declarar uma lista de inteiros assim:

```
list<int> minhaLista;
```

Listas

Para inserir elementos:

No final: `minhaLista.push_back(10);` // Insere 10 no final

No início: `minhaLista.push_front(5);` // Insere 5 no início

Remover elementos:

Do início: `minhaLista.pop_front();` // Remove o primeiro elemento

Do final: `minhaLista.pop_back();` // Remove o último elemento

Listas

Percorrer a lista:

```
for (int valor : minhaLista) {  
    cout << valor << " ";  
}  
cout << endl;
```

Para saber quantos elementos estão na lista:

```
cout << "Tamanho: " << minhaLista.size() << endl;
```

Listas

Exemplo de uso do método size:

```
list<int> c1;  
c1.push_back(5);  
cout << "List length is " << c1.size() << "." << endl;  
c1.push_back(7);  
cout << "List length is now " << c1.size() << "." << endl;
```

Para remover todos os elementos com valor igual a 10:

```
minhaLista.remove(10);
```

Observações – Lista

- A lista do C++ permite inserção e remoção eficiente em qualquer posição, mas o acesso direto (por índice) é lento.
- Para aceder um elemento específico, é necessário percorrer a lista até ele.

Filas (Queues)

- Definição: Estruturas que seguem o princípio **FIFO** (First-In, First-Out), onde o **primeiro elemento inserido é o primeiro a ser removido**.
- Aplicações: Sistemas de agendamento, buffers de dados, processamento de tarefas.
- Vantagens:
 - Gerenciamento eficiente de tarefas em ordem de chegada.
 - Simplicidade na implementação de algoritmos de processamento sequencial.
- Exemplo em C++: A STL fornece a classe **std::queue**, que encapsula o comportamento de uma fila.

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> fila;

    // Inserir elementos (enqueue)
    fila.push(10);
    fila.push(20);
    fila.push(30);

    // Aceder o elemento da frente
    cout << "Frente da fila: " << fila.front() << endl;

    // Remover elementos (dequeue)
    fila.pop();
    cout << "Nova frente: " << fila.front() << endl;

    // Tamanho da fila
    cout << "Tamanho da fila: " << fila.size() << endl;

    // Esvaziar a fila
    while (!fila.empty()) {
        cout << "Removendo: " << fila.front() << endl;
        fila.pop();
    }

    return 0;
}
```

Implementação manual com lista encadeada

```
#include <iostream>
using namespace std;

struct No {
    int valor;
    No* prox;
};

struct Fila {
    No* inicio;
    No* fim;
    int tamanho;

    Fila() : inicio(nullptr), fim(nullptr), tamanho(0) {}

    void enfileira(int v) {
        No* novo = new No{v, nullptr};
        if (fim) fim->prox = novo;
        else inicio = novo;
        fim = novo;
        tamanho++;
    }
};
```

```
void desenfileira() {
    if (inicio) {
        No* temp = inicio;
        inicio = inicio->prox;
        delete temp;
        tamanho--;
        if (!inicio) fim = nullptr;
    }
}

int frente() {
    if (inicio) return inicio->valor;
    throw "Fila vazia!";
}

bool vazia() { return inicio == nullptr; }
};
```

Implementação manual com lista encadeada

```
int main() {  
    Fila fila;  
    fila.enfileira(5);  
    fila.enfileira(10);  
    fila.enfileira(15);  
  
    cout << "Frente: " << fila.frente() << endl;  
    fila.desenfileira();  
    cout << "Nova frente: " << fila.frente() << endl;  
  
    while (!fila.vazia()) {  
        cout << "Remover: " << fila.frente() << endl;  
        fila.desenfileira();  
    }  
  
    return 0;  
}
```

Principais operações de fila

OPERAÇÃO	DESCRIÇÃO	EXEMPLO EM C++ PADRÃO
push	Insere elemento no final da fila	<code>fila.push(valor);</code>
pop	Remove elemento da frente da fila	<code>fila.pop();</code>
front	Consulta o elemento da frente	<code>fila.front();</code>
size	Retorna o número de elementos	<code>fila.size();</code>
empty	Verifica se a fila está vazia	<code>fila.empty();</code>

Pilhas (Stacks)

- Definição: Estruturas que seguem o princípio **LIFO (Last-In, First-Out)**, onde o **último elemento inserido é o primeiro a ser removido**.
- Aplicações: Algoritmos de recursão, análise de expressões, controle de execução.
- Vantagens:
 - Controle simples de fluxos de execução e reversão de operações.
 - Útil para resolver problemas que exigem retrocesso (backtracking).
- Exemplo em C++: A STL disponibiliza a classe **std::stack**, que implementa uma pilha.

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> pilha;

    // Inserir elementos (push)
    pilha.push(10);
    pilha.push(20);
    pilha.push(30);

    // Tamanho da pilha
    cout << "Tamanho da pilha: " << pilha.size() << endl;

    // Aceder ao topo
    cout << "Topo da pilha: " << pilha.top() << endl;

    // Remover elemento do topo (pop)
    pilha.pop();

    cout << "Novo topo: " << pilha.top() << endl;
    cout << "Tamanho após pop: " << pilha.size() << endl;

    // Esvaziar a pilha
    while (!pilha.empty()) {
        cout << "Removendo: " << pilha.top() << endl;
        pilha.pop();
    }

    return 0;
}
```

Implementar pilha manualmente com array

```
#include <iostream>
#define TAMANHO 5
using namespace std;

typedef struct {
    int topo;
    int itens[TAMANHO];
} PILHA;

void iniciaPilha(PILHA &p) {
    p.topo = -1;
}

bool pilhaVazia(PILHA p) {
    return p.topo == -1;
}

bool pilhaCheia(PILHA p) {
    return p.topo == TAMANHO - 1;
}
```

```
void empilha(PILHA &p, int x) {
    if (!pilhaCheia(p))
        p.itens[++p.topo] = x;
    else
        cout << "Pilha cheia!" << endl;
}

int desempilha(PILHA &p) {
    if (!pilhaVazia(p))
        return p.itens[p.topo--];
    else {
        cout << "Pilha vazia!" << endl;
        return -1;
    }
}

int main() {
    PILHA s;
    iniciaPilha(s);

    empilha(s, 5);
    empilha(s, 10);

    cout << "Desempilhado: " << desempilha(s) << endl;
    cout << "Desempilhado: " << desempilha(s) << endl;

    return 0;
}
```


Principais operações de uma pilha

OPERAÇÃO	DESCRIÇÃO	EXEMPLO EM C++ PADRÃO
push	Insere elemento no topo	<code>pilha.push(valor);</code>
pop	Remove elemento do topo	<code>pilha.pop();</code>
top	Consulta o elemento do topo	<code>pilha.top();</code>
size	Retorna o número de elementos	<code>pilha.size();</code>
empty	Verifica se a pilha está vazia	<code>pilha.empty();</code>

Observações – Pilha

- Pilhas são úteis para problemas como inversão de dados, avaliação de expressões e controle de chamadas de funções.
- A implementação manual permite maior controle, mas o uso de `std::stack` é mais prático para a maioria dos casos.

Recursos Avançados em C++

- **Templates:** Permitem criar listas, filas e pilhas genéricas, reutilizáveis para diferentes tipos de dados.
- **STL (Standard Template Library):** Oferece implementações otimizadas dessas estruturas, facilitando o desenvolvimento e aumentando a eficiência do código.

Comparação

ESTRUTURA	PRINCÍPIO	INSERÇÃO/REMOÇÃO	APLICAÇÕES TÍPICAS
Lista	Sequencial	Qualquer posição	Manipulação dinâmica de dados
Fila	FIFO	Início/Fim	Agendamento, buffers
Pilha	LIFO	Topo	Recursão, backtracking

Questionário



Exercicíos